

String Matching Algorithms

String Matching

Algorithms

- Naïve or Brute Force Search
- Knuth-Morris-Pratt Algorithm
- Longest Common subsequence
- Finite Automata Search

Introduction

- String matching or searching algorithms try to find places where one or several strings (also called patterns) are found within a larger string (searched text):

- **Text:**

... try to find places where one or several strings (also...

Pattern: ace

... try to find **places** where one or several strings (also...

APPLICATION

- Finding all occurrences of a pattern in a text is a problem that arises frequently in text-editing programs.
- To search for particular patterns in DNA sequences.

- Input description: A text string t of length n . A pattern string p of length m .
- Problem description: Find the first (or all) instances of the pattern in the text.

* You will always have my love,
my love, for the love I love is
lovely as love itself.*

love ?

* You will always have my love ,
my love , for the love I love
is love ly as love itself.*

INPUT

OUTPUT

Naïve string matching Algorithm

Naïve string matching Algorithm

- **String matching problem:** Given a text string t and a pattern string p , find all occurrences of p in t

- A naive algorithm for this problem simply

considers all possible starting positions i of a matching string within t , and compares p to the substring of t beginning at each such position I

Naive algorithm for Pattern Searching (Brute-Force Algorithm)

- The brute force algorithm consists in checking, at all positions in the text between 0 and $n-m$, whether an occurrence of the pattern starts there or not. Then, after each attempt, it shifts the pattern by exactly one position to the right.

- **Example** (1)

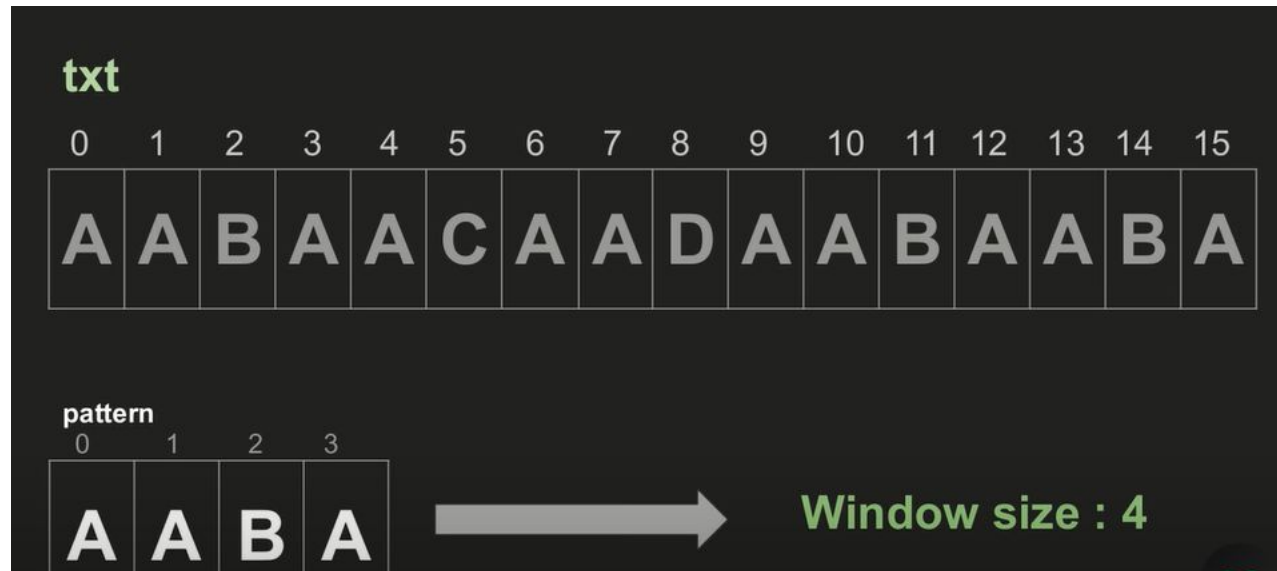
Input: txt[] =

"AABAACAADAABAABA" pat[]

= "AABA"

Output:

Naive algorithm for Pattern Searching (Brute-Force Algorithm)



Naive algorithm for Pattern Searching (Brute-Force Algorithm)

txt

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	A	B	A	A	C	A	A	D	A	A	B	A	A	B	A

txt

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	A	B	A	A	C	A	A	D	A	A	B	A	A	B	A

txt

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
A	A	B	A	A	C	A	A	D	A	A	B	A	A	B	A

Naive algorithm for Pattern Searching (Brute-Force Algorithm)

- **Example** (1)

Input: txt[] =

"AABAACAADAABAABA" pat[]

= "AABA"

Output: Pattern found at index 0

Pattern found at index 9

Pattern found at index 12

Text : A A B A A C A A D A A B A A B A

Pattern : A A B A

A	A	B	A							A	A	B	A		
A	A	B	A	A	C	A	A	D	A	A	B	A	A	B	A
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
												A	A	B	A

Pattern Found at 0, 9 and 12

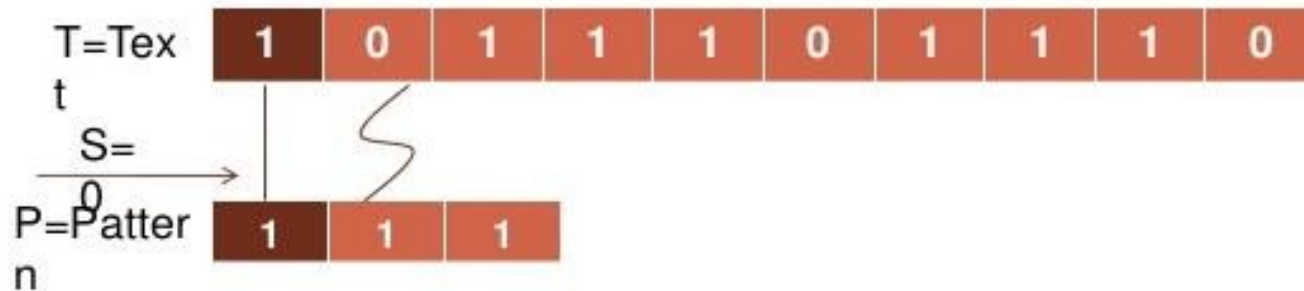
Example (2)

- SUPPOSE,

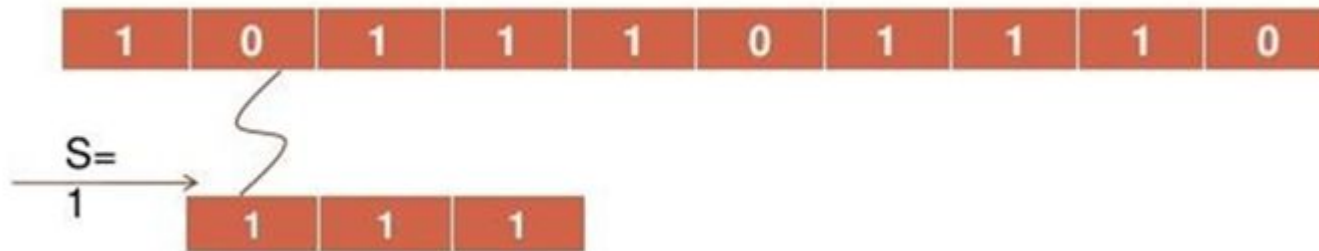
T=1011101110

P=111

FIND ALL VALID SHIFT.....



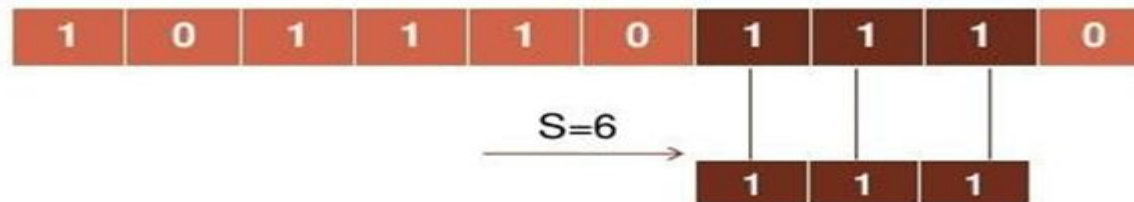
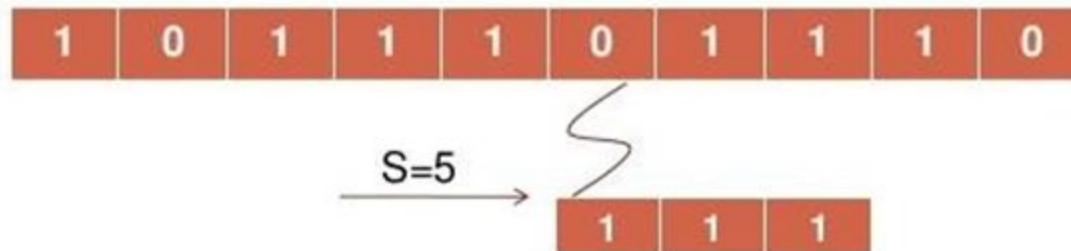
Example (2)



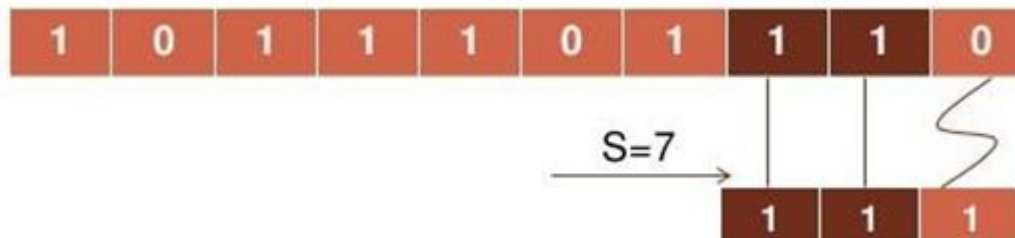
So, S=2 is a valid shift...



Example (2)



So, $S=6$ is a valid shift...



Main Features

- no preprocessing phase;
- **constant extra space needed;**
- always **shifts the window by exactly 1 position to the right;**
- comparisons can be done in any order;
- searching phase in **$O(mn)$** time complexity;

Algorithm

NAIVE-STRING-MATCHER (T, P)

1. $n \leftarrow \text{length } [T]$
2. $m \leftarrow \text{length } [P]$
3. for $s \leftarrow 0$ to $n - m$
4. do if $P[1 \dots m] = T[s + 1 \dots s + m]$
5. then print "Pattern occurs with shift" s

The naive algorithm finds all valid shifts using a loop that checks the condition $P[1 \dots m] = T[s+1 \dots s+m]$ for each of the $n - m + 1$ possible values of s .

(P =pattern , T =text/string , s =shift)

- **What is the worst case ?**

The worst case of Naive Pattern Searching occurs in following scenarios.

- 1) When **all characters of the text and pattern are same.**

txt[] = "AAAAAAAAAAAAAAAAAAAAA";

pat[] = "AAAAA";

- 2) Worst case also occurs when only the last character is different.

txt[] = "AAAAAAAAAAAAAAAAAAAAAB";

pat[] = "AAAAB";

The Knuth-Morris-Pratt (KMP) Algorithm

The Knuth-Morris-Pratt (KMP) Algorithm

- Knuth-Morris and Pratt introduce a linear time algorithm for the string matching problem.
- A matching time of **$O(n)$** is achieved by avoiding comparison with an element of 'S' that have previously been involved in comparison with some element of the pattern 'p' to be matched. i.e., backtracking on the string 'S' never occurs

Components of KMP Algorithm:

- 1. The Prefix Function (Π):** The Prefix Function, Π for a pattern encapsulates knowledge about how the pattern matches against the shift of itself. This information can be used to avoid a useless shift of the pattern 'p.' In other words, this enables avoiding backtracking of the string 'S.'
- 2. The KMP Matcher:** With string 'S,' pattern 'p' and prefix function ' Π ' as inputs, find the occurrence of 'p' in 'S' and returns the number of shifts of 'p' after which occurrences are found.

PREFIX- FUNCTION (P)

- Following pseudo code compute the prefix function, Π :

COMPUTE- PREFIX- FUNCTION (P)

1. $m \leftarrow \text{length } [P]$ // 'p' pattern to be matched
2. $\Pi [1] \leftarrow 0$
3. $k \leftarrow 0$
4. for $q \leftarrow 2$ to m
5. do while $k > 0$ and $P [k + 1] \neq P$
 $[q]$
6. do $k \leftarrow \Pi [k]$
7. If $P [k + 1] = P [q]$
8. then $k \leftarrow k + 1$
9. $\Pi [q] \leftarrow k$
10. Return Π

- **Running Time Analysis:**

- In the above pseudo code for calculating the prefix function, the for loop from step 4 to step 10 runs 'm' times.
- Step1 to Step3 take constant time. Hence the running time of computing prefix function is **$O(m)$** .

- **Example:** Compute Π for the pattern 'p' below:

P:

a	b	a	b	a	c	a
---	---	---	---	---	---	---

- **Solution**
- **Initially:** $m = \text{length } [p] = 7$
- $\Pi [1] = 0$
- $k = 0$

Step 1: $q = 2, k = 0$

$$\Pi[2] = 0$$

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
π	0	0					

Step 2: $q = 3, k = 0$

$$\Pi[3] = 1$$

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
π	0	0	1				

Step3: $q = 4, k = 1$

$$\Pi[4] = 2$$

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	A
π	0	0	1	2			

Step4: $q = 5, k = 2$

$$\Pi[5] = 3$$

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
π	0	0	1	2	3		

Step5: $q = 6, k = 3$

$$\Pi[6] = 0$$

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
π	0	0	1	2	3	0	

Step6: $q = 7, k = 1$

$$\Pi[7] = 1$$

q	1	2	3	4	5	6	7
p	a	b	a	b	a	c	a
π	0	0	1	2	3	0	1

- After iteration 6 times, the prefix function computation is complete:

q	1	2	3	4	5	6	7
p	a	b	A	b	a	c	a
π	0	0	1	2	3	0	1

The KMP

Matcher:

- The KMP Matcher with the pattern 'p,' the string 'S' and prefix function 'Π' as input, finds a match of p in S. Following pseudo code compute the matching component of KMP algorithm:

KMP-MATCHER (T, P)

1. $n \leftarrow \text{length}[T]$
2. $m \leftarrow \text{length}[P]$
3. $q \leftarrow 0$ // numbers of characters matched
4. $\Pi \leftarrow \text{COMPUTE-PREFIX-FUNCTION}(P)$
5. for $i \leftarrow 1$ to n // scan S from left to right
 6. do while $q > 0$ and $P[q + 1] \neq T[i]$
 7. do $q \leftarrow \Pi[q]$ // next character does not match
 8. If $P[q + 1] = T[i]$
 9. $q \leftarrow q + 1$ // next character matches
 9. then $q \leftarrow q + 1$ // is all of p matched?
10. If $q = m$ then print "Pattern occurs with shift" $i - m$
12. $q \leftarrow \Pi[q]$ // look for the next match

Running Time Analysis:

- The for loop beginning in step 5 runs 'n' times, i.e., as long as the length of the string 'S.' Since step 1 to step 4 take constant times, the running time is dominated by this for the loop. Thus running time of the matching function is **$O(n)$** .
- Example: Given a string 'T' and pattern 'P' as follows:

- Example (2)
- Let us execute the KMP Algorithm to find whether 'P' occurs in 'T.'

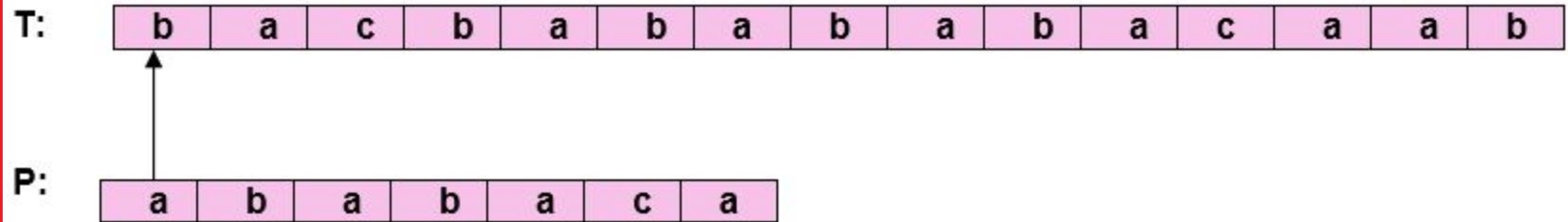
q	1	2	3	4	5	6	7
p	a	b	A	b	a	c	a
π	0	0	1	2	3	0	1

Solution:

Initially: $n = \text{size of } T = 15$
 $m = \text{size of } P = 7$

Step1: $i=1, q=0$

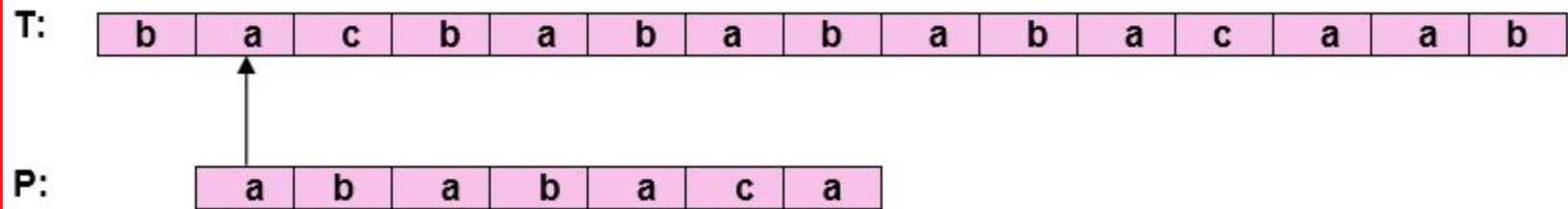
Comparing P [1] with T [1]



P [1] does not match with T [1]. 'p' will be shifted one position to the right.

Step2: $i = 2, q = 0$

Comparing P [1] with T [2]

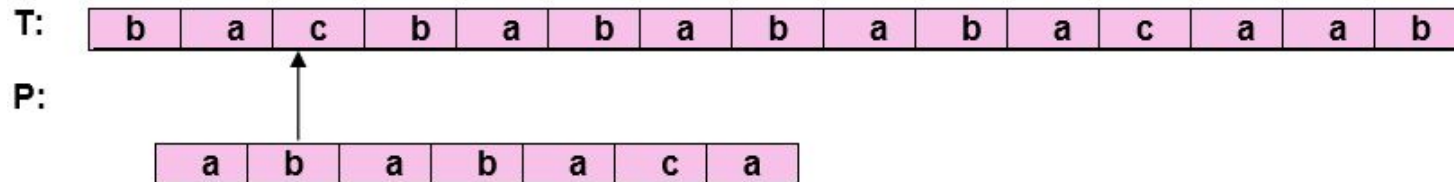


P [1] matches T [2]. Since there is a match, p is not shifted.

Step 3: $i = 3, q = 1$

Comparing P [2] with T [3]

P [2] doesn't match with T [3]

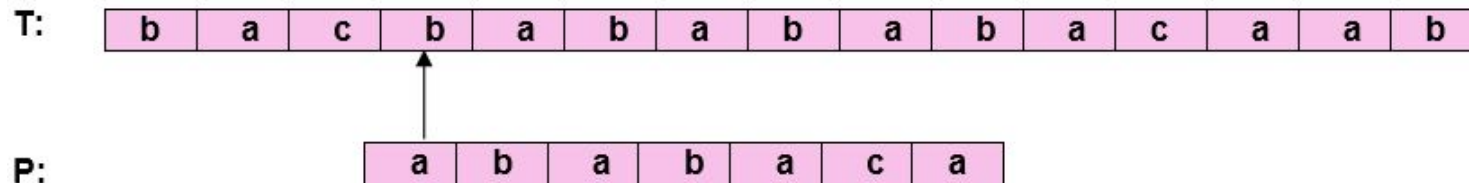


Backtracking on p, Comparing P [1] and T [3]

Step4: $i = 4, q = 0$

Comparing P [1] with T [4]

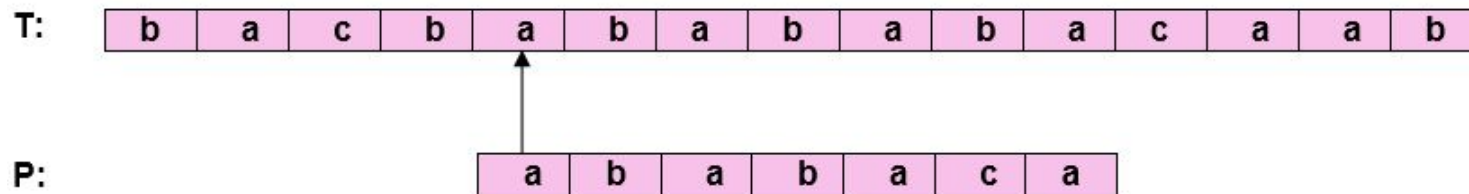
P [1] doesn't match with T [4]



Step5: $i = 5, q = 0$

Comparing P [1] with T [5]

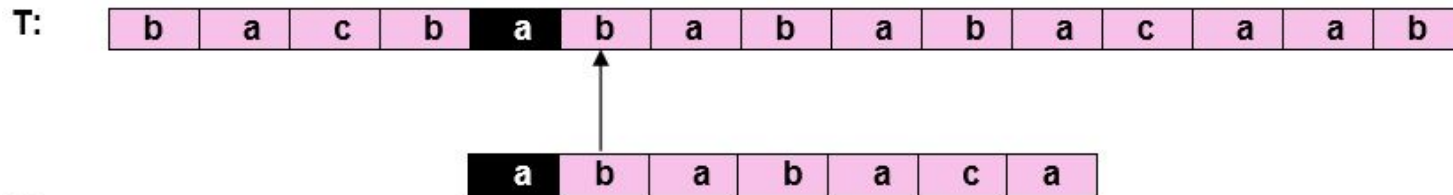
P [1] match with T [5]



Step6: $i = 6, q = 1$

Comparing P [2] with T [6]

P [2] matches with T [6]

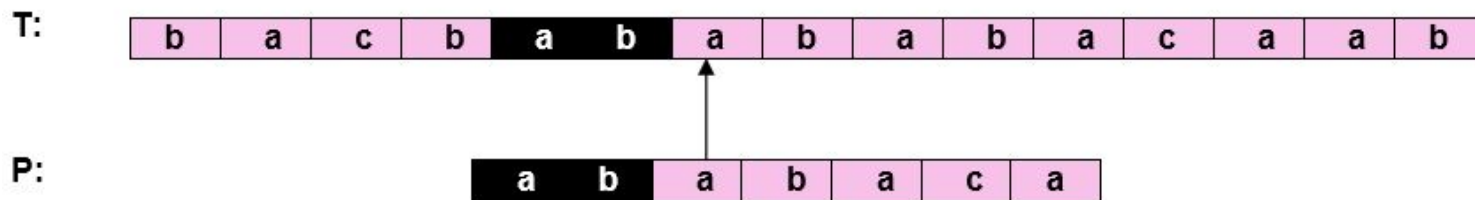


P:

Step7: $i = 7, q = 2$

Comparing P [3] with T [7]

P [3] matches with T [7]

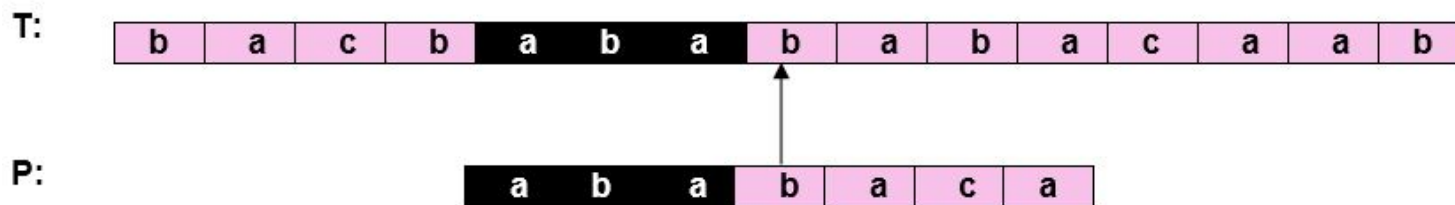


P:

Step8: $i = 8, q = 3$

Comparing P [4] with T [8]

P [4] matches with T [8]



P:

Step9: $i = 9, q = 4$

Comparing P [5] with T [9]

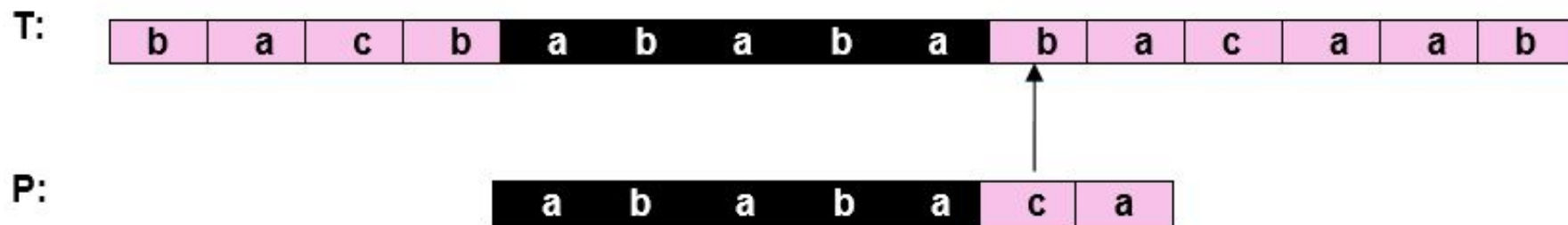
P [5] matches with T [9]



Step10: $i = 10, q = 5$

Comparing P [6] with T [10]

P [6] doesn't match with T [10]



Backtracking on p, Comparing P [4] with T [10] because after mismatch $q = \pi [5] = 3$

Step11: $i = 11, q = 4$

Comparing P [5] with T [11]

P [5] match with T [11]

T:

b	a	c	b	a	b	a	b	a	b	a	c	a	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

P:

a	b	a	b	a	c	a
---	---	---	---	---	---	---

Step12: $i = 12, q = 5$

Comparing P [6] with T [12]

P [6] matches with T [12]

T:

b	a	c	b	a	b	a	b	a	b	a	c	a	a	b
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

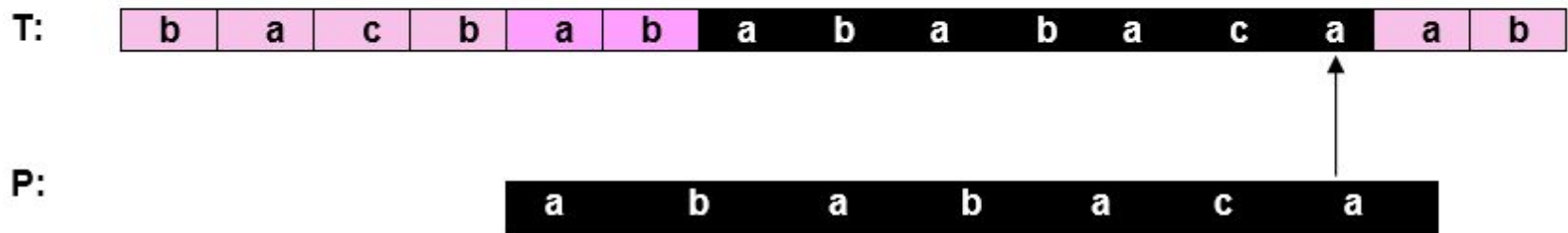
P:

a	b	a	b	a	b	a	c	a
---	---	---	---	---	---	---	---	---

Step13: $i = 3, q = 6$

Comparing P [7] with T [13]

P [7] matches with T [13]



Pattern 'P' has been found to complexity occur in a string 'T.' The total number of shifts that took place for the match to be found is $i - m = 13 - 7 = 6$ shifts.

Longest Common Subsequence Algorithm

Longest Common Subsequence Algorithm

- **PROBLEM DEFINITION:** A subsequence of a string S , is a set of characters that appear in left –to right order, but not necessarily consecutively.
- Example
- ACTTGCG
- • ACT , ATTC , T , ACTTGC are all subsequences.
- • TTA is not a subsequence
- • There are 2^n subsequences of string S of length n .

What is LCS?

- A common subsequence of two strings is a subsequence that appears in both strings. A longest common subsequence is a common subsequence of maximal length.

Example

S1 =

AAACCGTGAGTTATTCGTTCTAGAA S2

= CACCCCTAAGGTACCTTTGGTTC

- LCS is ACCTAGTACTTTG
- • Enumerate all sub-sequences of S1, and check if they are sub-sequences of S2. So the complexity will be $O(2^{nm})$

Algorithm

LCS – Length(X, Y)

m= length[X]

n= length[Y]

for i =1 to m

$c[i, 0] = 0$

for j =0 to n

$c[0, j] = 0$

for i = 1 to m

for j = 1 to n

if $x_i = y_j$

$c[i, j] = c[i - 1, j - 1] + 1$

$B[i, j] := 'D' \text{ or } \nwarrow$

else if $c[i - 1, j] > c[i, j - 1]$

$c[i, j] = c[i - 1, j]$

$B[i, j] := 'U' \text{ or } \uparrow$

else

$c[i, j] = c[i, j - 1]$

$B[i, j] := 'L' \text{ or } \rightarrow$

$\leftarrow$ return c and B

SEQUENCE RETRIEVAL

Algorithm: Print-LCS (B, X, i, j)

if $i = 0$ and $j = 0$

return

if $B[i, j] = 'D'$

 Print-LCS(B, X, $i-1, j-1$)

 Print(x_i)

else if $B[i, j] = 'U'$

 Print-LCS(B, X, $i-1, j$)

else

 Print-LCS(B, X, $i, j-1$)

Example:

- We have two strings $X = \text{ABCBDAB}$ and $Y = \text{BDCABA}$ to find the longest common subsequence. Following the algorithm LCS Length-

		j	0	1	2	3	4	5	6
		y_j		B	D	C	A	B	A
i	x_i								
0			0	0	0	0	0	0	0
1	A		0	↑	↑	↑	↖	←	↖
2	B		0	↖	←	←	↑	↖	←
3	C		0	↑	↑	↖	←	↑	↑
4	B		0	↖	↑	↑	↑	↖	←
5	D		0	↑	↖	↑	↑	↑	↑
6	A		0	↑	↑	↑	↖	↑	↖
7	B		0	↖	↑	↑	↑	↖	↑

Time complexity analysis

- Each call to subproblem takes constant time. We call it once from the main routine, and at most twice every time we fill in an entry of array L. There are $(m)(n)$ entries, so the total number of calls is at most $2(m)(n)$ and the time is $O(mn)$.
- As usual, this is a worst case analysis. The time might sometimes be better, if not all array entries get filled out. For instance if the two strings match exactly, we'll only fill in diagonal entries and the algorithm will be fast.

String Matching with Finite Automata



SOMAIYA
VIDYAVIHAR UNIVERSITY

K J Somaiya College of Engineering



- **Finite automata:**

- A finite state machine (FSM, also known as a deterministic finite automaton or DFA) is a way of representing a *language* (meaning a set of strings; we're interested in representing the set strings matching some pattern).

- A finite automaton is a quintuple $(Q, \Sigma, \delta, q_0, F)$:
- **Q**: the finite **set of states**
- **Σ** : the finite **input alphabet**
- **δ** : the “transition function” from $Q \times \Sigma$ to Q
- **$q_0 \in Q$** : the start state
- **$F \subset Q$** : the set of final (accepting) states

- The finite automaton begins in state q_0 and read the characters of its input string one at a time. If the automaton is in state q and reads input character a , it moves from state q to state $d(q,a)$.
- • As long as M is in a state belonging to A , M is said to have accepted the string read so far, an input that is not accepted is said to be rejected.

• FINITE AUTOMATA

- Finite Automata(FA) is the simplest machine to recognize patterns.

FA is characterized into two types:

- 1) **Deterministic Finite Automata (DFA)**
- 2) **Nondeterministic Finite Automata(NFA)**

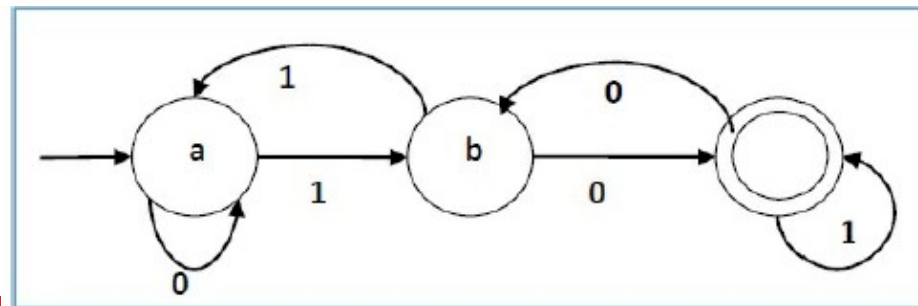
- A finite automaton accepts strings in a specific language. It begins in state q_0 and reads characters one at a time from the input string. It makes transitions (ϕ) based on these characters, and if when it reaches the end of the tape it is in one of the accept states, that string is accepted by the language.
- It has **finite number of states**
- Digraphs used to represent DFA is known as **state diagram**.
- It is useful for doing **lexical analysis** and **pattern matching**.
- ***There can be many possible DFAs for a pattern.*** A DFA with minimum number of states is generally preferred.

EXAMPLE

- Let a deterministic finite automata be
- $Q = \{a, b, c\}$, $\Sigma = \{0, 1\}$, $q_0 = \{a\}$, $F = \{c\}$, and
Transition function δ

Present State	Next State for Input 0	Next State for Input 1
a	a	b
b	c	a
c	b	c

State Diagram

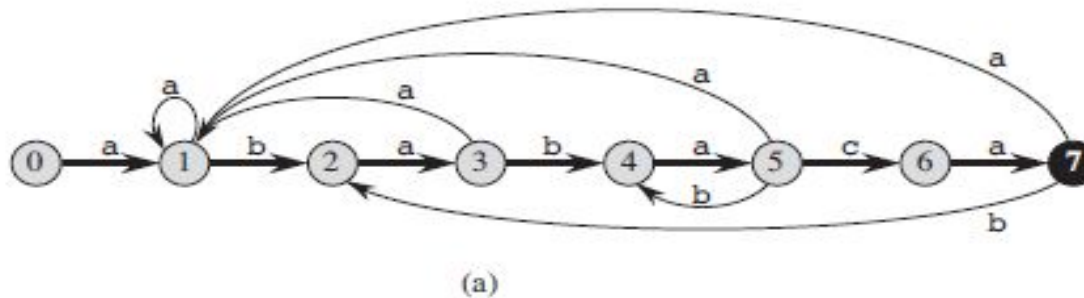


Example (2)

- The string-matching automaton is very efficient: it examines each character in the text exactly once and reports all the valid shifts in $O(n)$ time

32.3 String matching with finite automata

997



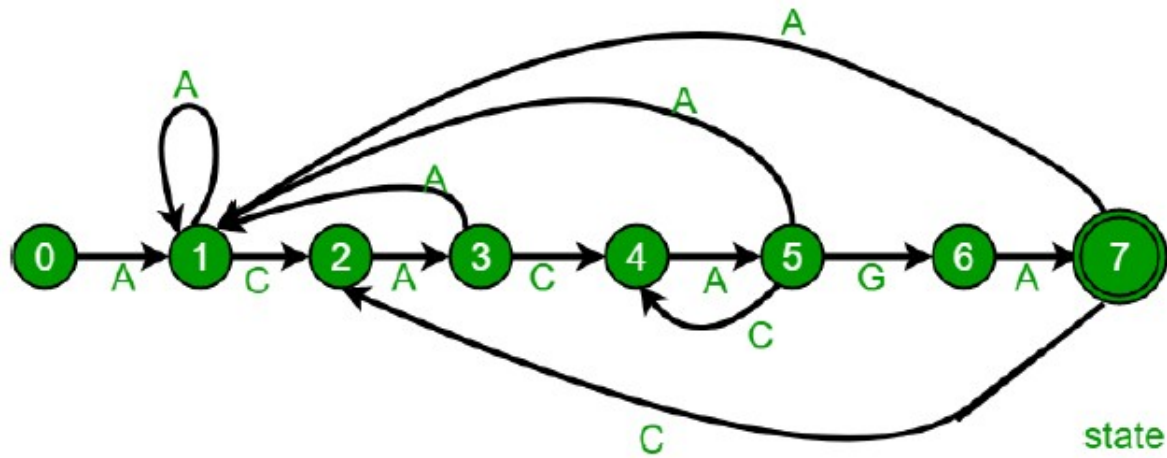
state	input			P
	a	b	c	
0	1	0	0	a
1	1	2	0	b
2	3	0	0	a
3	1	4	0	b
4	5	0	0	a
5	1	4	6	c
6	7	0	0	a
7	1	2	0	

(b)

i	—	1	2	3	4	5	6	7	8	9	10	11
T[i]	—	a	b	a	b	a	b	a	c	a	b	a
state $\phi(T_i)$	0	1	2	3	4	5	4	5	6	7	2	3

(c)

Example (3)



Pattern P=acacaga

state	character			
	A	C	G	T
0	1	0	0	0
1	1	2	0	0
2	3	0	0	0
3	1	4	0	0
4	5	0	0	0
5	1	4	6	0
6	7	0	0	0
7	1	2	0	0

- A state-transition diagram (previous slide) for the string-matching automaton that accepts all strings ending in the string ababaca.
- State 0 is the start state, and state 7 (shown blackened) is the only accepting state. A directed edge from state i to state j labeled a represents $(i,a) = j$.
- The right-going edges forming the "spine" of the automaton, shown heavy in the figure, correspond to successful matches between pattern and input characters.
- The left-going edges correspond to failing matches. Some edges corresponding to failing matches are not shown; by convention, if a state i has no outgoing edge labeled a for some a , then $(i,a) = 0$.
- (b) The corresponding transition function, and the pattern string $P = \text{ababaca}$.
- The entries corresponding to successful matches between pattern and input characters are shown shaded.
- (c) The operation of the automaton on the text $T = \text{abababacaba}$. Under each text character $T[i]$ is given the state $\delta(T_i)$ the automaton is in after processing the prefix T_i . One occurrence of the pattern is found, ending in position

BASIC IDEA

The basic idea is to build a automaton in which

- **Each character in the pattern has a state.**
- **Each match sends the automaton into a new state.**
- **If all the characters in the pattern has been matched, the automaton enters the accepting state.**
- **Otherwise, the automaton will return to a suitable state according to the current state and the input charactersuch thatthis returned state reflectsthe maximum advantage we can take from the previous matching.**
- **The matching takes $O(n)$ time since each character is examined once.**

Basic Idea

- To search a pattern $P = p_1p_2\dots p_m$ in a text $T = t_1t_2\dots t_n$. Number of states in FA will be $M+1$ where M is length of the pattern.
- State 0 will be the starting state, and state m will be the only accepting state.
- Get the next state from the current state for every possible character.
- We can get the next state by getting the length of the longest prefix of the given pattern such that the prefix is also suffix. The transition function chooses the next state to maintain the invariant: $\phi(T_i) = \sigma(T_i)$ After scanning in i characters, the state number is the longest prefix of P that is also a suffix of T_i .